FUNDACIÓN ARGENTINA DE SMALLTALK · 15/10/2026

protoST

The Smalltalk you know, from a file,
in places Smalltalk does not reach today.

Gustavo Marino

WHAT THIS IS

An experiment looking for enthusiasts.

protoST is not a better Smalltalk and does not compete with the live environment. It is something else: the code you already know how to write, running as just another program, on a core with native threads and other languages alongside.

Everything I show is in the repository and can be verified.

QUESTIONS, NOT ANSWERS

Sound familiar?

Deploy

What does it take to put an image behind something else as a service?

Cores

How many cores does your code use when the machine has twelve?

Integrate

How does it live alongside Python, JavaScript, queues, databases?

Share

How do you share code as text with someone who does not use an image?

WHY IT IS POSSIBLE NOW

protoCore: one core, several languages

Immutable

Collections that are "modified" by creating a new version that shares almost everything with the previous one.

Parallel

Native threads with no global lock and a concurrent garbage collector.

Shared

JavaScript, Python, Clojure, Scala and protoST on the same object model.

THE IDEA THAT MAKES IT SAFE

A mutable object is a reference to an immutable snapshot

`account`

atomic reference



```
{ owner: Ana balance: 120 }
```

snapshot: nobody changes it

A message to another thread carries a reference, not a copy, and every read sees a consistent snapshot: never a half-written state.

DEMO 1 · THE CODE YOU ALREADY KNOW

```
Account >> withdraw: amount
  amount > balance ifTrue: [
    ^ InsufficientFunds new signal: owner , ' has only ' , balance printString].
  balance := balance - amount

[attempts := attempts + 1. ana withdraw: 130]
  on: InsufficientFunds
  do: [:e | ana deposit: 50. e retry].
```

Classes, **printOn:**, non-local return, exceptions with **retry** and **resume:**, exact fractions. A text file; a blank line ends a method.

ACTORS

Any object can be an actor

```
c := Counter new asActor.
1 to: 1000 do: [:i | c increment].
c increment wait   "⟹ 1001"
```

Every message returns a **Future** at once (except identity, nil and printString, which the proxy answers).

The actor processes one message at a time: its state is plain instance variables, with no locks.

Actors share a pool of native threads.

DEMO 2 · REAL PARALLELISM

The same computation, on one thread and on twelve actors

```
futures := ranges collect: [:r |
  PrimeCounter new asActor countFrom: r first to: r last].
parallel := futures inject: 0 into: [:sum :f | sum + f wait]
```

Both results must agree (4946 primes); the times and the speedup are measured live.

IDEAS, NOT PROMISES

Worlds it could open up

Digital twins

One actor per physical component: each event is processed atomically and the components talk through messages.

Services

A process that starts in tens of milliseconds, reads files and standard input, speaks HTTP and TCP, and runs other programs: no image to deploy.

Other runtimes

Since protoCore 2.6, an embedded runtime reads and writes another's objects without copying them (tested: protoScala on protoST objects). The reverse is missing, and the binaries do not load each other yet.

DEMO 4 · A CONNECTED TWIN

Another program delivers the data; protoST processes and publishes it

```
server := HTTPServer on: 0 handler: [:request |
  (HTTPResponse ok: pump statusJson wait) contentType: 'application/json'].
log writeStreamDo: [:out |
  Stdio stdin linesDo: [:line |
    state := (pump reading: (JSON parse: line)) wait.
    out nextPutAll: line; lf]].
```

python3 04-connected-twin.feed | protost 04-connected-twin.st: JSON readings on standard input, the pump in an actor, its state over HTTP, and every reading in a file.

PERFORMANCE, WITH METHOD

Every benchmark verifies its result

Measurement	protoST	CPython
Startup (one expression)	25 ms	32–35 ms
One thread: the work, without startup (geomean)	14–20× slower	1.00
Twelve actors: pool against 1 worker	2.8×	—
saturation_big: 1→4 / 1→6 workers	2.5× / 2.9×, no ceiling up to 6	—

Source: benchmarks/reports/2026-10-12-protocore-2.14.6.md (protoST on protoCore 2.14.6; CPython 3.14 on the same machine). Synthetic, one laptop. From 4–7× (loops) to 30–35× (recursive dispatch) and 55–64× (exceptions). It is not fast: it is the same code on several cores, with no locks.

WHAT PROTOST IS NOT

All written down and verified

— No image or browser: the program is the file

— Immutable strings (WriteStream to build them)

— Thin metaclass; no thisContext

— A blank line ends a method

— Short symbols equal to strings

— An actor that is waiting does not handle other messages

Tutorial, chapter 14: what differs, why, and what to write instead.

HOW IT WAS VERIFIED

Adversarial reviews, before this talk

≈1100 audit programs

Written the way a Pharo programmer writes them, in two rounds; every silently wrong result was fixed with its own test.

A review of the I/O

3 critical and 8 important defects, fixed before the 0.5.0 release, with their regression tests.

1101 ctest cases

Not the same as the programs: conformance, unit, CLI, examples and documentation. CI runs them on Linux, macOS and Windows.

A rare hang

Caught with the process still alive: a lost wakeup in libstdc++ 13's std::counting_semaphore.

HOW TO TAKE PART

Write a program the way you would write it.

And tell me everything that surprises you.

`github.com/gamarino/protoST`

Packages for Ubuntu 24.04 and Windows (WSL2) under Releases · docs/TUTORIAL.md · chapter 14

DEMO 3 · DIGITAL TWIN (MODULE) · ABRIDGED OUTPUT

```
cycle 3  temp 72  flow 36  vib 3  running
cycle 4  temp 76  flow 48  vib 4  running
  temperature above 75: stopped
time per cycle: 56 ms (three 50 ms reads each)
```

A pump as a state machine in an actor; three sensors with 50 ms of simulated I/O, read in parallel on every cycle; an alarm stops the pump.

MODULE · INSIDE PROTOCORE

Cells, snapshots and a GC that starts early

64-byte cells

The size of a cache line; small integers inside the pointer.

Lists and ropes

Immutable AVL trees and ropes: a change costs O(log n) new nodes.

Concurrent GC

Starts each cycle before the ceiling and sweeps with several cursors and helper threads.

MODULE · OPEN QUESTIONS

What would be useful to you first?

- Runtimes loading each other (the core already allows it)
- thisContext and debugging tools
- Message-send performance
- More class-library protocol; WebSockets and HTTP/2